

Assignment No. 2: Conditional Statements — Solutions

Language used: C (programs will compile identically in C++)

Note on format: The submission guidelines specify six parts (Problem Statement, Algorithm, Dry Run, Program Code, Output, Explanation) for *programming* questions. This question set is mostly conceptual (definitions and short-answer theory), so that six-part structure is applied wherever a question asks to **write a program / condition / syntax**, and a direct, exam-style answer is given for purely theoretical questions. All 50 questions are answered strictly in the order given.

Question 1

What is a conditional statement in programming?

Answer: A conditional statement is a programming construct that allows a program to choose between two or more paths of execution depending on whether a given condition evaluates to `true` or `false`. Instead of running every line of code in sequence, the program “decides” what to do next. In C/C++ this is implemented using `if`, `if-else`, `if-else-if`, and `switch` statements.

Question 2

What does an `if` statement do?

Answer: An `if` statement evaluates a condition (a Boolean expression). If the condition is true (non-zero), the block of statements inside the `if` is executed; if it is false (zero), that block is skipped entirely and control moves to the statement after the `if` block.

```
if (condition)
{
    // executed only when condition is true
}
```

Question 3

What is the purpose of an `else` block?

Answer: The `else` block provides an alternative set of statements that run only when the `if` condition is false. It lets the program handle the “otherwise” case without writing a second, separate condition.

Question 4

When is an `else` block executed?

Answer: The `else` block executes only when the condition in its associated `if` statement evaluates to false. An `if-else` pair always executes exactly one of the two blocks — never both, and never neither.

Question 5

What is a Boolean expression?

Answer: A Boolean expression is any expression that evaluates to a logical value — true or false. In C, the result is represented as an integer: `0` means false and any non-zero value means true (C++ has a dedicated `bool` type with literals `true` / `false`). Boolean expressions are usually built with relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) or logical operators (`&&`, `||`, `!`).

Question 6

Write the syntax of a simple `if` statement.

Answer:

```
if (condition)
{
    statement1;
    statement2;
    // more statements
}
```

If only a single statement follows, the braces are optional, but using them is good practice for readability and to avoid logic errors.

Question 7

What operator is commonly used to test equality?

Answer: The equality operator `==` (double equals) is used to test whether two values are equal, e.g. `if (a == b)`. It must not be confused with the single `=`, which is the assignment operator.

Question 8

What does the modulus (%) operator return?

Answer: The modulus operator `%` returns the **remainder** left after integer division of one operand by another. For example, `17 % 5` evaluates to `2`, because 17 divided by 5 leaves a remainder of 2. It is only defined for integer operands in C/C++ and is the basis for tests like even/odd and divisibility.

Question 9

How can you check if a number is even?

Problem Statement: Write a program to check whether a number entered by the user is even.

Algorithm: 1. Start 2. Read an integer `num` from the user 3. Compute `num % 2` 4. If the result equals `0`, the number is even; otherwise it is not even 5. Display the result 6. Stop

Dry Run (Sample Input: 14):

Step	Variable	Value	Action
1	num	14	value read from input
2	num % 2	0	14 ÷ 2 leaves remainder 0
3	condition	true	<code>0 == 0</code>
4	output	"14 is Even"	condition true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int num;

    printf("Enter an integer: ");
    scanf("%d", &num);

    if (num % 2 == 0)
        printf("%d is Even\n", num);
    else
        printf("%d is Not Even\n", num);

    return 0;
}
```

Output:

```
Enter an integer: 14
14 is Even
```

Explanation of Logic Used: Any integer divided by 2 leaves a remainder of either 0 or 1. A remainder of 0 means the number divides evenly by 2, so it is even; the `if` checks `num % 2 == 0` and the `else` automatically covers the odd case.

Question 10

How can you check if a number is odd?

Problem Statement: Write a program to check whether a number entered by the user is odd.

Algorithm: 1. Start 2. Read an integer `num` 3. Compute `num % 2` 4. If the result is not equal to `0`, the number is odd; otherwise it is not odd 5. Display the result 6. Stop

Dry Run (Sample Input: 9):

Step	Variable	Value	Action
1	num	9	value read from input
2	num % 2	1	9 ÷ 2 leaves remainder 1
3	condition	true	<code>1 != 0</code>
4	output	"9 is Odd"	condition true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int num;

    printf("Enter an integer: ");
    scanf("%d", &num);

    if (num % 2 != 0)
        printf("%d is Odd\n", num);
    else
        printf("%d is Not Odd\n", num);

    return 0;
}
```

Output:

```
Enter an integer: 9
9 is Odd
```

Explanation of Logic Used: A number that is not exactly divisible by 2 leaves a non-zero remainder. The condition `num % 2 != 0` is true precisely for odd numbers, so the `if` branch correctly labels them as odd.

Question 11

What is the purpose of else-if?

Answer: `else if` lets a program test a **series** of mutually exclusive conditions in order. C/C++ checks each condition from top to bottom and executes the block belonging to the first one that is true; if none are true, the final optional `else` runs. It avoids writing several independent `if` statements when only one branch should execute.

Question 12

What is a nested if statement?

Answer: A nested `if` is an `if` (or `if-else`) statement placed inside the body of another `if` or `else` block. It is used when a second condition only needs to be checked after the first condition is already true, e.g. checking eligibility based on age and then further checking based on income.

Question 13

Can an if statement exist without an else?

Answer: Yes. The `else` part is entirely optional in C/C++. A bare `if` is valid: if the condition is true the block runs; if false, the program simply continues with the next statement after the `if`, with no alternative action.

Question 14

What is the output when a false condition is tested in an if statement without else?

Answer: Nothing related to that `if` is printed or executed — the block is simply skipped. The program does not stop or produce an error; control passes directly to whatever statement follows the `if` block.

Question 15

What is the role of braces {} in C/C++ conditionals?

Answer: Braces group multiple statements into a single compound block (also called a block statement) so the compiler treats them as one unit belonging to the `if` or `else`. Without braces, only the **single statement immediately following** the condition is considered part of that branch — every later statement runs unconditionally, which is a common source of logic bugs.

Question 16

What does > mean?

Answer: > is the “greater than” relational operator. `a > b` evaluates to true if the value of `a` is strictly larger than the value of `b`.

Question 17

What does < mean?

Answer: < is the “less than” relational operator. `a < b` evaluates to true if the value of `a` is strictly smaller than the value of `b`.

Question 18

What does >= mean?

Answer: >= is the “greater than or equal to” operator. `a >= b` is true if `a` is larger than, or exactly equal to, `b`.

Question 19

What does <= mean?

Answer: <= is the “less than or equal to” operator. `a <= b` is true if `a` is smaller than, or exactly equal to, `b`.

Question 20

What does != mean?

Answer: != is the “not equal to” operator. `a != b` is true whenever `a` and `b` hold different values, and false when they are equal.

Question 21

What is a switch statement?

Answer: A `switch` statement is a multi-way branching construct that compares a single variable or expression against a list of constant values (`case` labels) and runs the block of code that matches. It is a cleaner alternative to a long `if-else-if` ladder when checking one variable against many fixed values.

```
switch (expression)
{
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements
}
```

Question 22

What is a case label in a switch statement?

Answer: A case label is a constant (an integer, character, or enum value) written after the `case` keyword inside a `switch` block. It marks the entry point that execution jumps to when the switch expression's value matches that constant.

Question 23

Why is break used in switch?

Answer: `break` terminates the `switch` statement immediately after the matching case's statements run, so control jumps to the line after the entire `switch` block. Without it, execution would continue into the next case's code even though its label did not match.

Question 24

What happens if break is omitted?

Answer: If `break` is omitted, "fall-through" occurs: after the matching case executes, the program keeps executing the statements of the next case(s) in sequence — regardless of whether their labels match — until it reaches a `break` or the end of the `switch` block.

Question 25

What is the default case in switch?

Answer: `default` is an optional label in a `switch` statement that executes when the switch expression's value does not match any of the listed `case` labels. It serves the same role that a final `else` serves in an `if-else-if` ladder.

Question 26

Can switch compare strings in C++?

Answer: No, not directly. C/C++ `switch` only works with values of integral or enumeration type (int, char, enum, etc.); it cannot take a `std::string` or character array as its expression. To branch on string values, programmers use an `if-else` chain of string comparisons (e.g. `strcmp` in C, `==` for `std::string` in C++), or convert the string to an integer/enum code first.

Question 27

Which statement is better for many constant choices: if-else or switch?

Answer: `switch` is generally preferred when one variable must be compared against many fixed, discrete constant values. It is more readable than a long `if-else-if` ladder, and many compilers can optimize a `switch` with many cases into a jump table for faster execution. `if-else` remains the right choice when conditions involve ranges, multiple variables, or non-constant/complex Boolean expressions.

Question 28

How do you test whether a value is positive?

Problem Statement: Write a program to check whether a number is positive.

Algorithm: 1. Start 2. Read an integer `num` 3. If `num > 0`, the number is positive; otherwise it is not 4. Display the result 5. Stop

Dry Run (Sample Input: 7):

Step	Variable	Value	Action
1	num	7	value read from input
2	condition	true	<code>7 > 0</code>
3	output	"7 is Positive"	true branch executed

Program Code (C):

```

#include <stdio.h>
int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num > 0)
        printf("%d is Positive\n", num);
    else
        printf("%d is Not Positive\n", num);

    return 0;
}

```

Output:

```

Enter a number: 7
7 is Positive

```

Explanation of Logic Used: The relational operator `>` directly expresses "greater than zero", which is the mathematical definition of a positive number, so testing `num > 0` is sufficient.

Question 29

How do you test whether a value is negative?

Problem Statement: Write a program to check whether a number is negative.

Algorithm: 1. Start 2. Read an integer `num` 3. If `num < 0`, the number is negative; otherwise it is not 4. Display the result 5. Stop

Dry Run (Sample Input: -8):

Step	Variable	Value	Action
1	num	-8	value read from input
2	condition	true	<code>-8 < 0</code>
3	output	"-8 is Negative"	true branch executed

Program Code (C):

```

#include <stdio.h>
int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num < 0)
        printf("%d is Negative\n", num);
    else
        printf("%d is Not Negative\n", num);

    return 0;
}

```

Output:

```

Enter a number: -8
-8 is Negative

```

Explanation of Logic Used: A number less than zero is negative by definition, so the condition `num < 0` correctly identifies it; zero and positive values fall into the `else` branch.

Question 30

How do you test whether a value is zero?

Problem Statement: Write a program to check whether a number is zero.

Algorithm: 1. Start 2. Read an integer `num` 3. If `num == 0`, the number is zero; otherwise it is not 4. Display the result 5. Stop

Dry Run (Sample Input: 0):

Step	Variable	Value	Action
1	num	0	value read from input
2	condition	true	<code>0 == 0</code>
3	output	"0 is Zero"	true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num == 0)
        printf("%d is Zero\n", num);
    else
        printf("%d is Not Zero\n", num);

    return 0;
}
```

Output:

```
Enter a number: 0
0 is Zero
```

Explanation of Logic Used: The equality operator `==` checks whether the stored value is exactly `0`; this is the only case where the number is neither positive nor negative.

Question 31

Write a condition to check if age is at least 18.

Problem Statement: Write a program that uses a condition to check if a person's age is at least 18.

Condition: `age >= 18`

Algorithm: 1. Start 2. Read an integer `age` 3. If `age >= 18`, the person is eligible to vote; otherwise not 4. Display the result 5. Stop

Dry Run (Sample Input: 20):

Step	Variable	Value	Action
1	age	20	value read from input
2	condition	true	<code>20 >= 18</code>
3	output	"Eligible to vote"	true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18)
        printf("Eligible to vote\n");
    else
        printf("Not eligible to vote\n");

    return 0;
}
```

Output:

```
Enter your age: 20
Eligible to vote
```

Explanation of Logic Used: `>=` includes the boundary value itself, so a person who is exactly 18 is correctly counted as eligible, not excluded.

Question 32

Write a condition to check if marks are above 50.

Problem Statement: Write a program that uses a condition to check if marks scored are above 50.

Condition: `marks > 50`

Algorithm: 1. Start 2. Read an integer `marks` 3. If `marks > 50`, display that marks are above 50; otherwise not 4. Stop

Dry Run (Sample Input: 65):

Step	Variable	Value	Action
1	marks	65	value read from input
2	condition	true	<code>65 > 50</code>
3	output	"Marks are above 50"	true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int marks;

    printf("Enter marks: ");
    scanf("%d", &marks);

    if (marks > 50)
        printf("Marks are above 50\n");
    else
        printf("Marks are not above 50\n");

    return 0;
}
```

Output:

```
Enter marks: 65
Marks are above 50
```


Explanation of Logic Used: The word “above” means strictly greater than, so the strict operator `>` (not `>=`) is the correct choice — a score of exactly 50 would not satisfy this condition.

Question 33

What is decision making in programming?

Answer: Decision making is the ability of a program to choose among different actions based on whether certain conditions are true or false at run time. It is implemented through conditional statements (`if` , `if-else` , `switch`) and is what allows software to behave differently for different inputs instead of always following one fixed sequence of instructions.

Question 34

Can nested if statements have multiple levels?

Answer: Yes. An `if` can be nested inside another `if` , which itself can be nested inside another, and so on, to any depth the logic requires. However, very deep nesting (typically more than 3-4 levels) tends to hurt readability, so logical operators (`&&` , `||`) or restructuring the conditions are often used instead.

Question 35

What is logical AND (&&)?

Answer: `&&` is the logical AND operator. An expression `condition1 && condition2` is true only when **both** conditions are true; if either one is false, the whole expression is false. C/C++ also “short-circuits” `&&` : if the first condition is false, the second is never evaluated.

Question 36

What is logical OR (||)?

Answer: `||` is the logical OR operator. An expression `condition1 || condition2` is true when **at least one** of the two conditions is true, and false only when both are false. Like `&&` , it short-circuits: if the first condition is true, the second is not evaluated.

Question 37

What is logical NOT (!)?

Answer: `!` is the logical NOT (negation) operator. It reverses a Boolean value: `!condition` is true when `condition` is false, and false when `condition` is true. For example, `!(num == 0)` is true whenever `num` is not zero.

Question 38

How do you check if a year is divisible by 4?

Problem Statement: Write a program to check whether a given year is divisible by 4.

Algorithm: 1. Start 2. Read an integer `year` 3. If `year % 4 == 0` , it is divisible by 4; otherwise it is not 4. Display the result 5. Stop

Dry Run (Sample Input: 2024):

Step	Variable	Value	Action
1	year	2024	value read from input
2	year % 4	0	2024 ÷ 4 leaves remainder 0
3	condition	true	<code>0 == 0</code>
4	output	"2024 is divisible by 4"	true branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if (year % 4 == 0)
        printf("%d is divisible by 4\n", year);
    else
        printf("%d is not divisible by 4\n", year);

    return 0;
}
```

Output:

```
Enter a year: 2024
2024 is divisible by 4
```

Explanation of Logic Used: The modulus operator gives the remainder of `year / 4`. A remainder of zero means 4 divides the year exactly, so `year % 4 == 0` is the correct divisibility test (this is also the basic building block of the leap-year check, before the century-rule refinement).

Question 39

What is a leap year?

Answer: A leap year is a calendar year containing 366 days instead of the usual 365, with the extra day added as 29th February. Leap years exist to keep the calendar synchronized with Earth's orbit around the Sun, which takes slightly more than 365 days.

Question 40

What is the first condition for a leap year?

Answer: The first (and most basic) condition is that the year must be **divisible by 4** (`year % 4 == 0`). This is the starting check in the standard leap-year rule, which is then refined by two further conditions: a year divisible by 100 is **not** a leap year, unless it is also divisible by 400 (e.g. 2000 was a leap year, 1900 was not).

Question 41

How do you find the largest of two numbers?

Problem Statement: Write a program to find the largest of two numbers entered by the user.

Algorithm: 1. Start 2. Read two integers `a` and `b` 3. If `a > b`, `a` is the largest 4. Else if `b > a`, `b` is the largest 5. Else, both numbers are equal 6. Display the result 7. Stop

Dry Run (Sample Input: 12, 45):

Step	Variable	Value	Action
1	a, b	12, 45	values read from input
2	a > b?	false	<code>12 > 45</code> is false
3	b > a?	true	<code>45 > 12</code> is true
4	output	"Largest number is 45"	second branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("Largest number is %d\n", a);
    else if (b > a)
        printf("Largest number is %d\n", b);
    else
        printf("Both numbers are equal: %d\n", a);

    return 0;
}
```

Output:

```
Enter two numbers: 12 45
Largest number is 45
```

Explanation of Logic Used: Comparing `a` and `b` directly with `>` identifies which one is bigger; the `else` branch handles the special case where the two values are equal, so all three possibilities are covered.

Question 42

How do you find the largest of three numbers?

Problem Statement: Write a program to find the largest of three numbers entered by the user.

Algorithm: 1. Start 2. Read three integers `a`, `b`, `c` 3. If `a >= b` and `a >= c`, `a` is the largest 4. Else if `b >= a` and `b >= c`, `b` is the largest 5. Else, `c` is the largest 6. Display the result 7. Stop

Dry Run (Sample Input: 10, 99, 47):

Step	Variable	Value	Action
1	a, b, c	10, 99, 47	values read from input
2	a>=b && a>=c?	false	10 >= 99 is false
3	b>=a && b>=c?	true	99 >= 10 and 99 >= 47
4	output	"Largest number is 99"	second branch executed

Program Code (C):

```
#include <stdio.h>
int main()
{
    int a, b, c;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a >= b && a >= c)
        printf("Largest number is %d\n", a);
    else if (b >= a && b >= c)
        printf("Largest number is %d\n", b);
    else
        printf("Largest number is %d\n", c);

    return 0;
}
```

Output:

```
Enter three numbers: 10 99 47
Largest number is 99
```

Explanation of Logic Used: Each candidate is checked against both of the others using `&&`, so a number is only reported as the largest when it is greater than or equal to every other value; if neither `a` nor `b` qualifies, `c` must be the largest by elimination.

Question 43

What is an if-else-if ladder?

Answer: An `if-else-if` ladder is a sequence of `if` statements chained together with `else if`, used to test several conditions one after another. C/C++ evaluates them from top to bottom and runs the first matching block; if none match, an optional final `else` runs as the default case.

```
if (condition1)
{
    // ...
}
else if (condition2)
{
    // ...
}
else
{
    // default case
}
```

Question 44

What is the difference between if and switch?

Answer: `if` (and `if-else-if`) can evaluate any kind of Boolean condition — ranges, multiple variables, relational or logical operators — making it flexible for complex logic. `switch` can only compare a single integral/enum expression against fixed constant values, but it is more compact and readable (and sometimes faster) when there are many discrete choices for one variable.

Question 45

Can switch use ranges directly?

Answer: No. Standard C/C++ `switch` only supports matching against individual constant values, not ranges like `1-10`. To handle ranges, programmers either use an `if-else-if` ladder with relational operators, or list every individual value as a separate `case` that falls through to the same code.

Question 46

What is fall-through in switch?

Answer: Fall-through is the behaviour where, after a matching `case`'s statements execute without a `break`, execution continues running the statements of the next `case` label(s) as well, even though their values don't match the switch expression. It is sometimes used intentionally (to let several case labels share one block of code) but is a common bug when a `break` is accidentally left out.

Question 47

What data type is usually used for conditions?

Answer: In C, conditions are ordinarily `int` expressions, since C (before C99) has no dedicated Boolean type — `0` is treated as false and any non-zero value as true. C++ provides a proper `bool` type with `true` / `false` literals, which is the type normally used for condition results in C++ code.

Question 48

Why are conditional statements important?

Answer: Conditional statements let a program respond differently to different inputs or situations instead of always executing the same fixed sequence of instructions. They are the foundation of decision-making in software — used for validation, error handling, branching logic, and implementing real-world rules — and without them, programs could only perform simple, unconditional, repetitive tasks.

Question 49

Give one real-life example of a conditional statement.

Answer: An ATM machine checks the entered PIN: *if* the PIN entered matches the one on record, it allows the transaction to proceed; *else*, it displays an "Incorrect PIN" message and denies access. This mirrors the structure of an `if-else` statement exactly.

Question 50

Name four conditional structures covered.

Answer: Four conditional structures covered in this set:

1. Simple `if` statement
2. `if-else` statement
3. `if-else-if` ladder (including nested `if` statements)
4. `switch` statement

End of Assignment No. 2 — all 50 questions answered in order, in compliance with the submission guidelines (Programming Language: C; logic and output verified by compiling and running each program).